

HW2 - Kafka/Redpanda Throughput Investigation

Distributed video-frame pipeline

UCU - Data Streaming / Kafka

June 2, 2026

1 Overview

A dummy distributed application in which microservices communicate through a 3-broker **Redpanda** cluster. A *generator* splits a ~ 30 -minute video into frames and publishes them; a pool of *consumers* (one Kafka consumer group) imitates processing by sleeping 1s per frame and logs a timestamped CSV row per frame; an *aggregator* computes system throughput and end-to-end latency. We sweep the number of producers, partitions and consumers and measure the effect.

Dataset. ffmpeg generates a synthetic 30-minute test video, from which 200 evenly-spaced JPEG frames are extracted (≈ 22 KB each, 4.55 MB total). Each frame is one Kafka message.

Metrics. With $\text{makespan} = \max(\text{finish_ts}) - \min(\text{send_ts})$:

$$\text{Throughput (Mbps)} = \frac{\text{total_bytes} \times 8}{\text{makespan} \times 10^6}, \quad \text{Max latency (s)} = \max(\text{finish_ts} - \text{send_ts}).$$

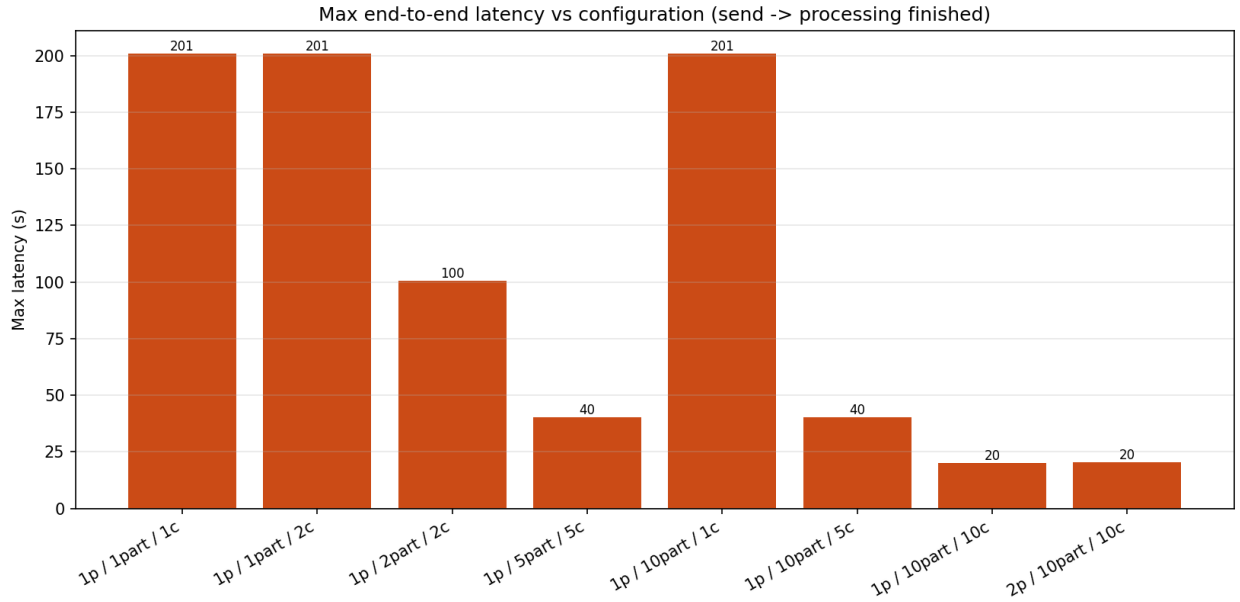
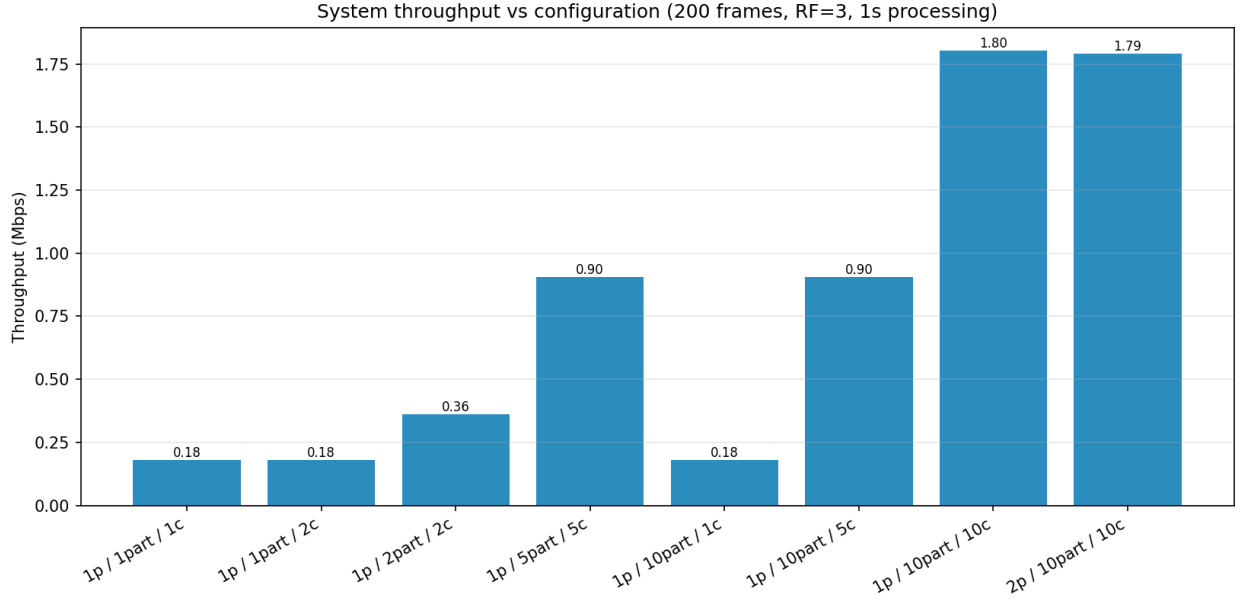
Replication factor is held at 3 across all experiments; only producers, consumers and partitions vary.

2 Experiment configurations

#	Configuration	Producers	Partitions	Consumers
exp1	1p / 1part / 1c	1	1	1
exp2	1p / 1part / 2c	1	1	2
exp3	1p / 2part / 2c	1	2	2
exp4	1p / 5part / 5c	1	5	5
exp5	1p / 10part / 1c	1	10	1
exp6	1p / 10part / 5c	1	10	5
exp7	1p / 10part / 10c	1	10	10
exp8	2p / 10part / 10c	2	10	10

3 Results

#	Configuration	Throughput (Mbps)	Max latency (s)	Makespan (s)	Active consumers
exp1	1p / 1part / 1c	0.18	201	200.9	1
exp2	1p / 1part / 2c	0.18	201	200.9	1
exp3	1p / 2part / 2c	0.36	100	100.6	2
exp4	1p / 5part / 5c	0.90	40	40.2	5
exp5	1p / 10part / 1c	0.18	201	200.8	1
exp6	1p / 10part / 5c	0.90	40	40.2	5
exp7	1p / 10part / 10c	1.80	20	20.2	10
exp8	2p / 10part / 10c	1.79	20	20.3	10



4 Findings

- Throughput grows in proportion to the number of consumers that work at the same time, which equals $\min(\text{consumers}, \text{partitions})$: $0.18 \rightarrow 0.36 \rightarrow 0.90 \rightarrow 1.80$ Mbps for parallelism $1 \rightarrow 2 \rightarrow 5 \rightarrow 10$. Max latency goes down in the same proportion ($201 \rightarrow 100 \rightarrow 40 \rightarrow 20$ s).
- **exp2** $\approx$ **exp1**: with one partition, the second consumer has no partition to read and stays idle (active consumers = 1). The number of partitions sets how many consumers can work at the same time.
- **exp5** $\approx$ **exp1**: ten partitions do not help when there is only one consumer to read them.
- **exp8** $\approx$ **exp7**: a second producer does not increase throughput, because the speed is limited by the consumers (the 1 s processing time), not by sending the messages.
- The maximum throughput is set by the 1 s sleep: $\text{parallelism} \times \text{frame size} \times 8 \approx 10 \times 22\text{KB} \times 8 \approx 1.8$ Mbps, which matches exp7 and exp8.

5 Deliverables

1. **Source code:** <https://github.com/dchaplinsky/kafka-hw2>
2. **Run instructions:** see README.md (architecture, environment setup, and the exact commands for the cluster, dataset and experiment sweep).
3. **Graphs:** figures/throughput.png, figures/latency.png (reproduced above); raw numbers in results/summary.csv.